

Week 2 - Day 7b Supplementary: More Recursion

Question 1: Tower of Hanoi [□□□]

You may have heard of the game "Tower of Hanoi" (see figure below):

The goal of the game is to move all the disks from the leftmost peg to the rightmost peg. The middle peg can be used as a helper peg to temporarily hold the disks.

There are only two simple rules:

1. You can only move one disk at a time.
2. You can only place a smaller disk on top of a larger disk.

The puzzle starts with n disks on the leftmost peg, and the disks are sorted from smallest to largest, with the largest disk on the bottom.

Your task is to write a **recursive function** called `hanoi(n, source, helper, target)` that prints out the steps to solve the Tower of Hanoi puzzle.

For example:

```
hanoi(1, "A", "B", "C")
# ---> prints "Move disk 1 from peg A to peg C"

hanoi(2, "A", "B", "C")
# ---> prints "Move disk 1 from peg A to peg B"
              "Move disk 2 from peg A to peg C"
              "Move disk 1 from peg B to peg C"

hanoi(3, "A", "B", "C")
# ---> prints "Move disk 1 from peg A to peg C"
              "Move disk 2 from peg A to peg B"
              "Move disk 1 from peg C to peg B"
              "Move disk 3 from peg A to peg C"
              "Move disk 1 from peg B to peg A"
              "Move disk 2 from peg B to peg C"
              "Move disk 1 from peg A to peg C"
```

Hint: Think about what the base case and recursive case would be; the recursive case should reduce the problem to a smaller version of itself (in this problem, a Tower of Hanoi with fewer disks).

```

# TODO: Implement the Towers of Hanoi function
def hanoi(n, source, helper, target):
    # Hint: the base case is implemented below for you
    if n == 0:
        return

    # TODO: your code for the recursive here
    # NOTE: you don't need to return anything from the function;
    # just print out the individual moves

# Test cases
for tower_height in range(1, 4):
    print('For', tower_height, 'disks:')
    hanoi(tower_height, 'A', 'B', 'C')
    print()

```

Question 2: Permute String [] [] [] []

Write a recursive function, `permute_string(s)`, that returns all possible permutations of the string `s`.

For example:

```

permute_string("ab") == ["ab", "ba"]
permute_string("abc") == ["abc", "acb", "bac", "bca", "cab", "cba"]

```

Implement `permute_string(s)` recursively.

```

# TODO: Implement a recursive function that returns all permutations
of a string
def permute_string(s):
    # your code here
    return []

# after you finish, these all should print true
print('Test passed:', set(permute_string("ab")) == set(["ab", "ba"]))
print('Test passed:', set(permute_string("abc")) == set(["abc", "acb",
"bac", "bca", "cab", "cba"]))
print('Test passed:', set(permute_string("aaa")) == set(["aaa"]))

```

Congratulations []